Pregunta 1

Se está desarrollando una aplicación para enviar notificaciones. Actualmente existe la siguiente clase:

```
class NotificacionService {

    public void enviar(Notificacion n, String tipo) {

        if (tipo.equals("EMAIL")) {
            ...
        } else if (tipo.equals("SMS")) {
            ...
        }

    }

}
```

El cliente solicita añadir próximamente notificaciones por WhatsApp, Telegram y Push.

¿Cuál sería la mejor refactorización?

A) Sustituir los `if` por un `switch`.

B) Crear una interfaz `CanalNotificacion` e implementar una clase para cada tipo de notificación.

C) Crear un método `enviarWhatsApp()` y otro `enviarTelegram()` dentro de la misma clase.

D) Convertir `NotificacionService` en una clase abstracta.

Pregunta 2

Una aplicación contiene la siguiente interfaz:

```
public interface Documento {

    void abrir();

    void guardar();

    void imprimir();

}
```

Posteriormente se implementa la clase:

```
class DocumentoSoloLectura implements Documento {

    public void abrir() { ... }

    public void guardar() {
        throw new UnsupportedOperationException();
    }

    public void imprimir() { ... }

}
```

¿Cuál es el principal problema del diseño?

A) Se incumple el principio de inversión de dependencias.

B) La interfaz debería dividirse en interfaces más específicas.

C) El método `guardar()` debería estar vacío en lugar de lanzar una excepción.

D) La interfaz debería ser una clase abstracta.

Pregunta 3

Observa el siguiente código:

```
class PedidoService {

    private MySQLRepository repository =
            new MySQLRepository();

    public void guardar(Pedido pedido) {
        repository.save(pedido);
    }

}
```

¿Cuál de las siguientes modificaciones mejora el diseño siguiendo SOLID?

A) Declarar `repository` como `static`.

B) Instanciar el repositorio dentro del método `guardar()`.

C) Sustituir `MySQLRepository` por una interfaz e inyectar la implementación.

D) Convertir `MySQLRepository` en Singleton.

Pregunta 4

Una empresa desarrolla una aplicación para calcular descuentos.

Inicialmente existe una única regla:

```
double calcular(Cliente cliente)
```

Con el tiempo aparecen descuentos para estudiantes, jubilados, clientes premium, campañas promocionales y empleados.

¿Cuál es el diseño más adecuado?

A) Añadir un `if` para cada nuevo tipo de descuento.

B) Crear un método distinto dentro de la misma clase para cada descuento.

C) Crear una clase diferente para cada regla de descuento implementando una misma interfaz.

D) Utilizar una única clase con un gran `switch`.

Pregunta 5

Se desea desarrollar una aplicación que pueda almacenar información tanto en MySQL como en PostgreSQL.

Se propone el siguiente diseño:

```
class ClienteService {

    private Repository repository;

    public ClienteService(Repository repository) {
        this.repository = repository;
    }

}
```

donde `Repository` es una interfaz implementada por `MySQLRepository` y `PostgreSQLRepository`.

¿Qué ventaja principal aporta este diseño?

A) Evita tener que utilizar excepciones.

B) Reduce el consumo de memoria de la aplicación.

C) Hace que las consultas SQL sean más rápidas.

D) Permite sustituir una base de datos por otra sin modificar `ClienteService`.